

Statistics 6910: Debugging Strategies

- Write code in a plain text file, e.g., in a script in RStudio
- Run the code in blocks in RStudio; debug as necessary
- OR: source() code into R (do not copy and paste)
- Syntax error will be caught and line number given
- Line numbers may not locate the error exactly
- Sometimes the error occurred earlier, but it gives you a starting place

Sourcing Code:

```
source("C://JUE//Teaching//Stat6910_Fa2013_AdvancedR//
LectureNotes/L06b_BuggyCode.R")
```

Error in

```
source("C://JUE//Teaching//Stat6910_Fa2013_Advanced
R//LectureNotes/L06b_BuggyCode.R") :
```

```
  C://JUE//Teaching//Stat6910_Fa2013_AdvancedR//
  LectureNotes/L06b_BuggyCode.R:9:28: unexpected ')
```

```
8: mybug = function(L, lambda) {
```

```
9:   numHits = rpois(L*lambda)
```

```
      ^
```

In RStudio:

```
mybug = function(L, lambda) {
  numHits = rpois(L*lambda)
  hitLocs = runif(0, numHits)
  return(numHits, hitLocs)
}
```

Error: unexpected ')' in:

```
"mybug = function(L, lambda) {
  numHits = rpois(L*lambda)"...
```

Can you see the error? What is wrong? What else is wrong ...?

Some Debugging Strategies

1. The `traceback` function prints the sequence of calls that led to the last error. This **may** show you where in your function something is going wrong.

It may not even be in the function itself, but in another function that is being called within the original function.

```
cv <- function(x) sd(x/mean(x))
cv()
Error in x/mean(x) : 'x' is missing
traceback()
4: is.data.frame(x)
3: var(if (is.vector(x)) x else as.double(x), na.rm = na.rm)
2: sd(x/mean(x)) at #1
1: cv()
```

2. If you have some idea where the error is occurring, you can use `print` to check that key variables are what you think they are.

3. Consider “commenting out” lines of your code where the error might occur, then adding them back in one by one.

4. To step through the function, expression by expression, and be able to print out any variable at each step, use the `debug` function. Use `undebug` to turn off debugging. The `browser` function works similarly.

While in the debugging mode, you can use the following commands:

'n' (or just return) - Advance to the next step.
 'c' - continue to the end of the current context: e.g., to the end of the loop if within a loop or to the end of the function.
 'Q' - exit the debugging mode and the current evaluation and return to the top-level prompt.

You can also evaluate any valid R expression. For example, you can type the names of variables to see their current values.

Efficient Programming

The first rule of efficient programming in R is to make use of vectorized calculations and the `apply` mechanisms whenever possible.

You can check how much time it takes to evaluate any expression by wrapping it in `system.time()`. Units are in seconds.

```
system.time(normal.samples <- rnorm(10000000))
user system elapsed
2.15 0.03 2.17
```

← wall clock time

CPU time for R process CPU time for system on behalf of R